

آشنایی با RPC و فناوری های مختلف آن

1. تعریف RPC :

RPC یک سازوکار ارتباطی در سیستم های توزیع شده است که به برنامه ها اجازه می دهد توابع یا روال هایی (Procedures) را که روی ماشین های دیگر قرار دارند، فراخوانی کنند.

2. ایده اصلی

1. شفافیت دسترسی (Access Transparency): برخلاف روش های سنتی که در آن برنامه نویس

باید صراحتاً از دستورات ارسال و دریافت پیام (send و receive) استفاده کند، در RPC جزئیات پیام رسانی از دید برنامه نویس پنهان می ماند.

2. عملکرد مشابه فراخوانی محلی: وقتی فرآیندی در ماشین A تابعی را در ماشین B فراخوانی

می کند، فرآیند فراخوان در ماشین A متوقف (Suspend) می شود و اجرای تابع در ماشین B صورت می گیرد.

3. انتقال داده ها: اطلاعات از طریق پارامترها به ماشین مقصد فرستاده می شوند و نتایج نیز در

قالب خروجی تابع به ماشین مبدأ باز می گردند.

3. تفاوت های فراخوانی محلی و فراخوانی از راه دور

1. محل اجرا

در فراخوانی محلی، تابع/رویه روی همان ماشین و در همان فضای آدرس اجرا می شود.

در RPC، فراخواننده و فراخوانده روی ماشین های متفاوت هستند.

2. انتقال پارامترها

در محلی، پارامترها مستقیماً توسط زبان/Runtime مدیریت می شوند.

در RPC، پارامترها باید توسط client stub بسته بندی (marshal) شوند و در سمت سرور توسط server stub باز شوند.

3. شفافیت

فراخوانی محلی طبیعی و ساده است

RPC سعی می کند مثل یک فراخوانی محلی به نظر برسد، یعنی شفاف باشد؛ اما این شفافیت کامل نیست.

4. مدل خطا

در محلی، خطاها ساده ترند چون همه چیز روی یک ماشین است.

در RPC، ممکن است یکی از ماشین‌ها یا ارتباط شبکه دچار خرابی شود، پس مدیریت خطا پیچیده‌تر است.

4. نقش client stub , server stub :

Client Stub نسخه‌ای از تابع/رویه است که به برنامه‌ی کلاینت داده می‌شود تا مثل یک فراخوانی محلی به نظر برسد. پارامترها را بسته‌بندی کند، پیام را برای سرور بفرستد، منتظر پاسخ بماند، نتیجه را باز کند و به فراخواننده برگرداند.

Server Stub معادل سمت سرور Client Stub است. پیام رسیده از شبکه را بگیرد، پارامترها را باز کند، تابع واقعی سرور را صدا بزند، نتیجه را بسته‌بندی کند و آن را به کلاینت برگرداند

5. مفهوم serialization , deserialization :

1. serialization : این فرآیند به معنی تبدیل پارامترهای یک فراخوانی (که در حافظه به صورت ساختارهای داده‌ای هستند) به یک قالب پیام (Message Format) است که بتوان آن را از طریق شبکه ارسال کرد. این کار توسط Client Stub انجام می‌شود. استاب کلاینت پارامترها را برمی‌دارد و آن‌ها را به شکلی بسته‌بندی می‌کند که برای انتقال در شبکه مناسب باشد.

2. این فرآیند دقیقاً برعکس مرحله قبل است؛ یعنی باز کردن پیام دریافتی از شبکه و تبدیل دوباره آن به پارامترهایی که برای برنامه قابل فهم باشد. این کار توسط Server Stub (هنگام دریافت درخواست) و دوباره توسط Client Stub (هنگام دریافت نتیجه از سرور) انجام می‌شود.

این فرآیند به دلیل ناهم‌هنگی (Heterogeneity) ماشین‌ها ضروری است. ممکن است یک ماشین از ترتیب بایت‌های Big-endian استفاده کند و دیگری از Little-endian. ممکن است نمایش اعداد اعشاری یا کاراکترها در دو سیستم متفاوت باشد.

6. مفهوم IDL:

برای اینکه کلاینت و سرور بتوانند با هم صحبت کنند، باید بر سر یک رابط (Interface) توافق داشته باشند. IDL زبانی است که برای توصیف این رابط به کار می‌رود؛ بدون اینکه به جزئیات پیاده‌سازی (مثلاً اینکه کد به زبان C یا Java نوشته شده) کاری داشته باشد. یکی از کاربردهای اصلی IDL که در کتاب به آن اشاره شده، تولید خودکار کد است: برنامه‌نویس رابط تابع (اسم تابع، پارامترهای ورودی و خروجی) را در یک فایل IDL تعریف می‌کند. سپس یک کامپایلر IDL این فایل را می‌خواند و به صورت خودکار Client Stub و Server Stub را تولید می‌کند.

این کار باعث می‌شود برنامه‌نویس درگیر پیچیدگی‌های Marshaling و باز و بسته کردن پیام‌ها نشود.

IDL اجازه می‌دهد کلاینت و سرور با زبان‌های مختلف نوشته شوند. برای مثال، یک کلاینت که به زبان ++C نوشته شده می‌تواند با سروری که به زبان Java است ارتباط برقرار کند، چون هر دو از یک تعریف رابط یکسان (IDL) پیروی می‌کنند.

IDL به ما اجازه می‌دهد مشخص کنیم یک سرویس «چه کاری» انجام می‌دهد، بدون اینکه بگوییم «چگونه» آن را انجام می‌دهد. این موضوع برای حفظ شفافیت (Transparency) ضروری است.

7. خطاهای رایج در RPC:

1. Timeout :

این رایج‌ترین خطا در RPC است. زمانی رخ می‌دهد که کلاینت درخواستی را ارسال می‌کند اما در بازه زمانی تعیین‌شده، پاسخی از سرور دریافت نمی‌کند.

2. Server Unavailable :

در این حالت، کلاینت سعی می‌کند به سرور متصل شود، اما سرور پاسخگو نیست.

3. Interface/Version Mismatch :

عدم تطابق IDL

4. Serialization/Marshalling Errors :

برای انتقال داده در شبکه، اشیاء باید به فرمت باینری یا متنی (مثل JSON یا Protobuf) تبدیل شوند.

5. Network Partition/Failure :

گاهی اوقات مشکل نه از کلاینت است و نه از سرور، بلکه مسیر ارتباطی دچار مشکل شده است.

6. Resource Exhaustion :

زمانی رخ می‌دهد که سرور زنده است اما توان پردازش درخواست جدید را ندارد.

7. Authentication & Authorization

8. تفاوت RPC با REST

1. RPC: بر پایه عملیات (Actions) است. در RPC شما یک تابع یا متد را فراخوانی می‌کنید (مثل

getUser()).

REST: بر پایه منابع (Resources) است. در REST شما با یک موجودیت تعامل دارید (مثل /users/123).

2. RPC: دارای Tight Coupling است. کلاینت باید دقیقاً بداند چه متدهایی در سرور وجود دارد و پارامترهای آنها چیست. تغییر در سمت سرور معمولاً مستلزم تغییر در کلاینت است.

REST: دارای Loose Coupling است. به دلیل استاندارد بودن متدها (GET, POST, etc) و ساختار منابع، کلاینت و سرور استقلال بیشتری دارند.

3. REST: به صورت بومی از سیستم Caching پروتکل HTTP پشتیبانی می‌کند. پاسخ‌های متد GET به راحتی توسط مرورگرها یا CDNها ذخیره می‌شوند.

RPC: به دلیل اینکه اکثر درخواست‌ها با متد POST ارسال می‌شوند یا ماهیت "فراخوانی متد" دارند، قابلیت کش‌شدن بسیار دشواری دارد.

9.

موارد استفاده رایج	پشتیبانی از streamin	مناسب برای سیستم توزیع شده	سادگی پیاده سازی	کارایی و سربار ارتباطی	پشتیبانی از چند زبان	روش تعریف رابط	فرمت داده	
میکروسرویس ها	بله (دوطرفه)	بله	متوسط	بالا	C++, Java, Python, Go, C# (.NET Core), Node.js, Ruby, PHP, Dart (برای Flutter), Kotlin, Objective-C	IDL	Binary	gRPC
ارتباط	محدود	بله	متوسط	بالا	C++, Java,	IDL	Binary/Compact	Thrift

			Python, PHP, Ruby, # C Erlang, Haskell, Perl, Smalltalk, OCaml, Delphi, Rust, Go, Swift					داخلی سرور ها
JSON-RPC	JSON	ندارد	هر زبانی که بتواند متن (JSON یا XML) را پردازش کند و از پروتکل HTTP پشتیبانی کند، می تواند از آنها استفاده کند	متوسط	ساده	در ابعاد کوچک	خیر	بلاکچین

سؤالات تحلیلی

(۱) چرا RPC باعث می شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی به نظر برسد این شباهت به دلیل استفاده از مفهومی به نام Stub (در سمت کلاینت) و Skeleton (در سمت سرور) است. RPC جزئیات پیچیده شبکه (مانند باز کردن سوکت، سریال سازی داده ها و ارسال پکت ها) را درون یک تابع محلی در سمت کلاینت (Client Stub) پنهان می کند.

(۲) چرا این شباهت می تواند خطرناک یا گمراه کننده باشد؟
تلاش برای مخفی کردن خطاها (Masking failures) می تواند سیستم را کندتر کند. مثلاً اگر یک RPC به طور مداوم سعی کند با سروری که موقتاً در دسترس نیست ارتباط برقرار کند تا خطا را از

کلاینت مخفی نگه دارد، این کار باعث معطلی بیش از حد کاربر می‌شود. در حالی که گاهی بهتر است سیستم سریعاً شکست را بپذیرد و به کاربر اجازه دهد تصمیم دیگری بگیرد.

(۳) چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد

(۱) Packet Loss : داده‌ها در میان راه ناپدید شوند

(۲) Network Partition : کلاینت و سرور هر دو سالم هستند اما نمی‌توانند با هم حرف بزنند.

(۳) Timeout : کلاینت بیش از حد منتظر پاسخ می‌ماند.

(۴) سرور درخواست را می‌گیرد اما قبل از ارسال پاسخ خاموش می‌شود.

(۵) عدم تطابق نسخه : کلاینت و سرور از نسخه‌های متفاوتی از رابط (Interface) استفاده کنند.

(۴) در چه شرایطی gRPC انتخاب بهتری از REST است

(۱) میکروسرویس‌های داخلی: وقتی تعداد زیادی سرویس در یک دیتاستر با هم در ارتباط هستند و سرعت اولویت اول است.

(۲) نیاز به Streaming: زمانی که نیاز به ارسال مداوم داده‌ها داریم (مثل ارسال دیتای زنده سنسورها یا چت تصویری).

(۳) سیستم‌های با محدودیت پهنای باند: به دلیل فشرده‌سازی باینری (Protobuf)، gRPC حجم دیتای بسیار کمتری نسبت به JSON در REST منتقل می‌کند.

(۴) ارتباطات چندزبانه (Polyglot): وقتی می‌خواهیم با یک فایل قرارداد (proto.) به طور خودکار برای چندین زبان مختلف (مثل Go و Python) کد تولید کنیم.

(۵) در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تری است

(۱) API‌های عمومی (Public API): وقتی می‌خواهید توسعه‌دهندگان دیگر به راحتی از سرویس شما استفاده کنند (REST برای همه شناخته شده است و نیاز به نصب ابزار خاصی ندارد).

(۲) کش‌شدن (Caching): زمانی که می‌خواهید از قابلیت کشینگ مرورگرها و CDN‌ها برای کاهش بار سرور استفاده کنید (REST به طور بومی از کش HTTP پشتیبانی می‌کند).

(۳) ارتباط مستقیم با مرورگر (Frontend): اکثر مرورگرها و فریم‌ورک‌های جاوااسکریپت با REST و JSON بسیار راحت‌تر از پروتکل‌های باینری مثل gRPC تعامل دارند.